

# Mantiqiy Tizimlarda Maqsad va Muammoni Shakllantirishning Amaliy Jihatlar

## 1. Kirish: Maqsadga Yo'naltirilgan Agentlar va Muammoni Hal Qilish Tamoyillari

Sun'iy intellekt (SI) tizimlari, yoki agentlar, o'z atrof-muhitini idrok etish va muayyan maqsadlarga erishish uchun harakatlarni amalga oshirishga mo'ljallangan. Bu jarayonda muammo va maqsadni aniq belgilash markaziy rol o'ynaydi. Muammoni to'g'ri shakllantirish, agentning samaradorligi, aniqligi va kutilgan natijalarga erishish qobiliyatini kafolatlash uchun hal qiluvchi ahamiyatga ega. SI agenti uchun vazifani samarali hal etish, uning maqsadli xulq-atvorini tartibga soluvchi asosiy tamoyillarni tushunishni talab qiladi.

Bu tamoyillar ko'pincha "Shakllantirish, Qidirish, Ijro Etish" (Formulate, Search, Execute) deb nomlanuvchi uch bosqichli oddiy dizayn paradigmasi sifatida taqdim etiladi. Bu model muammoni hal qiluvchi har qanday agentning asosiy arxitekturasini tashkil etadi:

- Maqsadni shakllantirish (Goal Formulation):** Agent avval o'zining joriy holatiga asoslanib, erishmoqchi bo'lgan maqsadli holatini aniqlaydi. Bu, agentning harakatlarini tor doirada belgilash va optimallashtirish imkonini beradi. Masalan, Ruminiyaning Arad shahrida joylashgan agent uchun Buxarestga yetib borish maqsad sifatida shakllantirilishi mumkin.
- Muammoni shakllantirish (Problem Formulation):** Maqsad aniqlangandan so'ng, agent bu maqsadga erishish uchun zarur bo'lgan kompyuter muammosini tuzadi. Bu bosqichda, agentning harakatlari, atrof-muhitdagi holatlar va ular o'rtasidagi bog'liqliklar aniq belgilanadi.
- Qidirish (Search):** Agent muammoni hal qilish uchun harakatlarning optimal ketma-ketligini topish maqsadida qidiruv algoritmini ishga tushiradi. Bu jarayon agentga mavjud muhit haqidagi ma'lumotlardan foydalanib, gipotetik harakatlar va ularning natijalarini ko'rib chiqishga imkon beradi.
- Ijro etish (Execution):** Yechim topilgandan so'ng, agent topilgan harakatlar ketma-ketligini birin-ketin bajaradi. Agent ushbu bosqichda oldindan topilgan yechimga amal qiladi va vazifa bajarilgandan so'ng yangi maqsadni shakllantirib, jarayonni takrorlaydi.

Bu paradigмага muvofiq, muammoni shakllantirishning o'zi ham bir qator atrof-muhit xususiyatlariga, masalan, uning statik yoki dinamikligi, kuzatilishi mumkinligi (fully observable) va deterministik yoki nondeterministikligiga bog'liq bo'ladi. Ushbu jihatlar muammoning murakkabligini belgilaydi va mos keladigan yechim usullarini tanlash uchun asos bo'ladi.

## 2. Maqsad va Muammoning Rasmiy Shakllantirilishi

Sun'iy intellektda samarali yechimlarni topish jarayoni real dunyo muammosini kompyuter tizimi hal qila oladigan aniq va rasmiy muammoga aylantirishdan boshlanadi. Bu jarayon muammoning asosiy komponentlarini belgilashni talab qiladi, bu esa AI agentiga o'z atrof-muhitini tushunish, mumkin bo'lgan harakatlarni baholash va oldindan belgilangan maqsadga erishish uchun harakat qilish imkonini beradi.

### 2.1. Muammoni aniqlash: Mavjud Vaziyat va Maqsadli Natija

Maqsad va muammo tushunchalari ko'pincha bir-biriga bog'liq bo'lsa-da, ular o'rtasidagi farqni tushunish juda muhim. **Maqsad (Goal)** – bu agent erishmoqchi bo'lgan yakuniy holat yoki natija. Masalan, avtonom transport tizimi uchun "yukni

B manziliga yetkazish" maqsad bo'lishi mumkin. Bu maqsadning o'zi agentga yo'nalish beradi va uning muvaffaqiyatini baholash uchun mezonni belgilaydi.

**Muammo (Problem)** esa bu maqsadga erishish uchun tuzilgan rasmiy computational modeldir. Bu model boshlang'ich nuqtani, mumkin bo'lgan harakatlarni va maqsadga erishish shartlarini belgilaydi.

Real-dunyo muammolarini aniq shakllantirishda "5 Nima Uchun?" (5 Whys) kabi amaliy usullar yordam berishi mumkin. Ushbu usul dastlabki muammo bayonotidan boshlab, uning asosiy sababini topish uchun "Nima uchun?" degan savolni ketma-ket takrorlashni o'z ichiga oladi. Masalan, "AI vositasi kutilgan natijani bermayapti" muammosidan boshlab, savollar ketma-ketligi "Nima uchun? — Chunki so'rov (prompt) yetarlicha aniq emas" va keyingi savolga o'tish "Nima uchun so'rov noaniq?" kabi jarayon orqali muammoning ildiziga yetib borishga yordam beradi. Bu usul AI agentni tahlil qila oladigan aniq va yo'naltirilgan muammoni shakllantirishda muhim ahamiyatga ega.

## 2.2. Muammoning Asosiy Rasmiy Komponentlari

Muammo to'g'ri shakllantirilganda, u beshta asosiy rasmiy komponentdan iborat bo'ladi :

1. **Boshlang'ich Holat (Initial State):** Agentning faoliyatini boshlaydigan dastlabki nuqta yoki shart-sharoitlar. Bu, agentning muammoni hal qilish jarayonini qayerdan boshlashini belgilaydi.
2. **Harakatlar To'plami (Actions Set):** Berilgan holatda agent amalga oshirishi mumkin bo'lgan barcha mumkin bo'lgan harakatlarning tavsifi. Bu to'plam, shuningdek, "voris funksiyasi" (successor function) deb ham ataladi, chunki u harakatlar orqali keyingi holatlarga o'tish imkonini beradi.
3. **Maqsadli Sinov (Goal Test):** Bu funksiya berilgan holatning maqsad holati ekanligini tekshiradi. U muammoni hal qilish jarayonining qachon yakunlanishini belgilaydi va agentga o'z muvaffaqiyatini baholash imkonini beradi.
4. **Yo'l Narxi Funksiyasi (Path Cost Function):** Bu funksiya boshlang'ich holatdan maqsadga erishish uchun ketma-ket harakatlar yo'lining umumiy narxini o'lchaydi. Narx vaqt, masofa, yoqilg'i sarfi yoki boshqa amaliy mezonlar bilan o'lchanishi mumkin. Agent eng past narxli yechimni topishga intiladi.

Ushbu komponentlarni tushunish va amalda qo'llashni osonlashtirish uchun bir nechta klassik misollar mavjud:

- **Vakuumli Dunyo (Vacuum World):** Bu oddiy, ammo o'rganish uchun samarali modelda :
  - **Boshlang'ich Holat:** Agentning joylashuvi va xonalarning toza yoki iflos holati.
  - **Harakatlar:** "Chapga harakatlanish", "o'ngga harakatlanish" va "so'rish".
  - **Maqsadli Sinov:** Har ikkala xona ham toza bo'lgan holatga erishildi.
  - **Yo'l Narxi:** Har bir harakatning narxi (masalan, 1 birlik).
- **Sakkizta Feruza Muammosi (8-Queens Problem):** Bu muammoda feruzalarni shaxmat taxtasiga shunday joylashtirish kerakki, ularning hech biri bir-biriga hujum qilmasin. Muammoning murakkabligi va yechim makonining kattaligi, muammoni rasmiy shakllantirishning muhimligini ko'rsatadi.

Ushbu nazariy tushunchalarni amaliy misollar bilan birlashtirish quyidagi jadvalda ko'rsatilgan.

| Komponent                  | Ta'rif                                                  | Vakuumli Dunyo Misoli                                  | Sakkizta Feruza Muammosi Misoli                                           |
|----------------------------|---------------------------------------------------------|--------------------------------------------------------|---------------------------------------------------------------------------|
| <b>Boshlang'ich Holat</b>  | Agentning boshlang'ich nuqtasi.                         | Agent A xonada, A iflos, B iflos.                      | Bo'sh shaxmat taxtasi.                                                    |
| <b>Harakatlar To'plami</b> | Berilgan holatda mumkin bo'lgan harakatlar.             | {Chapga harakatlanish, O'ngga harakatlanish, So'rish}. | {Feruza (i, j) joyga qo'yish}.                                            |
| <b>Maqsadli Sinov</b>      | Maqsad holatiga erishilganligini tekshirish funksiyasi. | Barcha xonalar toza holatga kelgan.                    | Taxtada 8 ta feruza bo'lib, ularning hech biri bir-biriga hujum qilmaydi. |
| <b>Yo'l Narxi</b>          | Yo'lning umumiy narxini o'lchash.                       | Har bir harakat 1 birlik narxga ega.                   | Odatda harakatlar soni bilan o'lchanadi.                                  |

### 3. Qidiruvning Nazariy Asoslari: Holat Maydoni va Qidiruv Ob'ektlari

Muammo rasmiy shakllantirilgandan so'ng, agentning keyingi vazifasi maqsadli holatga olib boradigan harakatlar ketma-ketligini topishdir. Bu jarayon **holat maydoni (state space)** deb nomlanuvchi nazariy tushunchaga asoslanadi. Holat maydoni – bu boshlang'ich holatdan barcha mumkin bo'lgan harakatlar ketma-ketligi orqali erishish mumkin bo'lgan barcha holatlar to'plamidir. Ushbu maydon ko'pincha yo'naltirilgan graf sifatida tasvirlanadi, unda tugunlar (nodes) holatlarni, qirralar (edges) esa harakatlarni ifodalaydi.

Qidiruv jarayonini vizuallashtirish va tushunish uchun **qidiruv daraxti (search tree)** kontseptsiyasidan foydalaniladi. Bu daraxt ildiz tugunida boshlang'ich holat bilan boshlanadi, harakatlar esa shoxlar orqali yangi holatlarni (bolalar tugunlari) hosil qiladi. Har bir qidiruv tuguni, aslida, muammoning ma'lumotlarini saqlovchi struktura bo'lib, u quyidagi komponentlarga ega bo'lishi mumkin:

- **Holat:** Tugun mos keladigan holat.
- **Ota-tugun (Parent-node):** Joriy tugunni yaratgan oldingi tugun.
- **Harakat (Action):** Ota-tugundan joriy tugunga o'tish uchun qo'llanilgan harakat.
- **Yo'l Narxi (Path-cost):** Boshlang'ich holatdan joriy tugunga bo'lgan yo'lning narxi.
- **Chuqurlik (Depth):** Ildiz tugundan joriy tugungacha bo'lgan yo'lning uzunligi.

#### Qidiruv Daraxtlari va Qidiruv Graflari O'rtasidagi Farq: Samaradorlikka Ta'siri

Qidiruv graflari va qidiruv daraxtlari tushunchasi ko'pincha chalkashliklarga sabab bo'ladi. Asosiy farq shundaki, daraxt (tree) o'z tabiatiga ko'ra hech qanday siklga ega bo'lmagan grafning (graph) maxsus turidir. Shu sababli, "daraxt qidiruvi" (tree search) algoritmi bitta holatga ikki marta tashrif buyurmaslikni kafolatlaydigan mexanizmlarni o'z ichiga olmaydi, chunki daraxtlarda bu kafolat tabiiy ravishda mavjud.

Aksincha, real dunyodagi muammolar holat maydoni deyarli har doim sikllarga ega bo'lgan umumiy grafdir. Bu holatda, agar daraxt qidiruvi qo'llanilsa, u cheksiz tsikllarga tushib qolishi

yoki bir xil holatlarni qayta-qayta tekshirishi mumkin, bu esa hisoblash samaradorligini keskin pasaytiradi. Bunday vaziyatda

"**graf qidiruvi**" (**graph search**) algoritmi qo'llaniladi. Graf qidiruvi algoritmi allaqachon tekshirilgan tugunlarni *yopiq ro'yxat* (closed list) deb nomlanuvchi ma'lumot tuzilmasida saqlaydi va shu bilan takroriy harakatlarning oldini oladi.

Bu shuni anglatadiki, algoritmlarning ishlash mantig'i ularning asosiy taxminlariga bog'liq. Daraxt qidiruvi takroriy tugunlar mavjud emas deb taxmin qiladi va shuning uchun bu muammoni hal qilishga mo'ljallangan qo'shimcha mexanizmlarni o'z ichiga olmaydi. Graf qidiruvi esa bu muammoning mavjudligini biladi va uni *yopiq ro'yxat* orqali samarali hal qiladi. Shu sababli, graf qidiruvi algoritmidan xotira sarfi yuqoriroq bo'ladi, chunki u tekshirilgan barcha tugunlarni saqlashi kerak. Bu sabab-oqibat munosabati, har bir algoritmnin qo'llashda o'ziga xos xotira va tezlik almashinuvlari (trade-offs) mavjudligini ko'rsatadi.

## 4. Umumiy Qidiruv Algoritmлари

Qidiruv algoritmлари ma'lum bir muammo uchun yechim topish maqsadida holat maydonini tizimli ravishda o'rganish mexanizmlaridir. Har qanday qidiruv algoritmining markaziy elementi **qidiruv chegarasi** (fringe yoki frontier) hisoblanadi. Chegara – bu hozirda kengaytirilishga tayyor bo'lgan (ya'ni, tekshirilishi kerak bo'lgan) barcha tugunlarning to'plamidir. Algoritmлар o'rtasidagi asosiy farq, navbatdagi qaysi tugunni kengaytirish uchun chegaradan tanlab olish usuli bilan aniqlanadi.

### 4.1. Ma'lumotsiz (Uninformed) Qidiruv Usullari

Ma'lumotsiz qidiruv algoritmлари, yechimga yo'l haqida qo'shimcha ma'lumot (evristika) dan foydalanmaydi. Ular faqatgina holat maydonini muntazam ravishda o'rganish orqali ishlaydi.

- **Kenglik bo'ylab qidiruv (BFS - Breadth-First Search):**

- **Tamoyil:** BFS qidiruvni ildiz tugundan boshlab, darajama-daraja (level-by-level) kengaytiradi. U barcha "qo'shni" tugunlarni o'rganib chiqqandan so'nggina keyingi darajaga o'tadi.
- **Ma'lumot Tuzilmasi:** BFS navbat (Queue) ma'lumot tuzilmasidan foydalanadi. Navbat

**FIFO** (First In, First Out - Birinchi kirgan, birinchi chiqadi) tamoyilida ishlaydi, bu esa ildizga eng yaqin joylashgan tugunlarning birinchi bo'lib tekshirilishini ta'minlaydi.

- **Qo'llanilishi:** Bu algoritm eng qisqa yo'lni topish uchun ideal hisoblanadi (agar barcha harakatlarning narxi bir xil bo'lsa), chunki u maqsadga eng yaqin joylashgan tugunni birinchi bo'lib topadi.

- **Chuqurlik bo'ylab qidiruv (DFS - Depth-First Search):**

- **Tamoyil:** DFS qidiruvni ildiz tugundan boshlab, bitta yo'l bo'ylab imkon qadar chuqur boradi. Agar "o'lik oxir" ga (dead end) yetsa, orqaga qaytib (backtracking) boshqa yo'lni qidirishni boshlaydi.
- **Ma'lumot Tuzilmasi:** DFS stek (Stack) ma'lumot tuzilmasidan foydalanadi. Stek

LIFO (Last In, First Out - Oxirgi kirgan, birinchi chiqadi) tamoyilida ishlaydi, bu esa eng so'nggi qo'shilgan (ya'ni, hozirgi chuqur yo'ldagi) tugunning birinchi bo'lib tekshirilishini ta'minlaydi.

- **Qo'llanilishi:** Bu algoritm yechimlar ildizdan uzoqda bo'lgan muammolar uchun ko'proq mos keladi va labirint, jumboq o'yinlari kabi "backtracking" talab qilinadigan vaziyatlar uchun juda qulay.

## 4.2. Algoritmnlarni Taqqoslash va Amaliy Tanlov

Navbat (Queue) va stek (Stack) oddiy ma'lumot tuzilmalari emas, balki ular qidiruv strategiyasini to'g'ridan-to'g'ri belgilaydigan mexanizmlardir. Navbatning FIFO xususiyati darajama-daraja kenglik bo'ylab qidiruvni ta'minlaydi, chunki u har doim eng eski tugunni (ya'ni, ildizga eng yaqin tugunni) kengaytiradi. Boshqa tomondan, stekning LIFO xususiyati bir yo'l bo'ylab chuqur qidiruvni ta'minlaydi, chunki u har doim eng yangi (ya'ni, chuqur darajada joylashgan) tugunni kengaytiradi. Bu sabab-oqibat munosabati, har bir algoritmni qo'llashda asosiy omil hisoblanadi.

| Parametrlar                   | Kenglik bo'ylab Qidiruv (BFS)                | Chuqurlik bo'ylab Qidiruv (DFS)            |
|-------------------------------|----------------------------------------------|--------------------------------------------|
| <b>Ma'lumot Tuzilmasi</b>     | Navbat (Queue)                               | Stek (Stack)                               |
| <b>Tamoyil</b>                | FIFO (First In, First Out)                   | LIFO (Last In, First Out)                  |
| <b>Xotira sarfi</b>           | Ko'proq xotira talab qiladi.                 | Kamroq xotira talab qiladi.                |
| <b>Tezlik</b>                 | DFS'ga nisbatan sekinroq.                    | BFS'ga nisbatan tezroq.                    |
| <b>Maqsadga mosligi</b>       | Maqsad manbaga yaqin bo'lganda samarali.     | Maqsad manbadan uzoqda bo'lganda samarali. |
| <b>Cheksiz tsikllar xavfi</b> | Cheksiz tsiklga tushib qolish muammosi yo'q. | Cheksiz tsiklga tushib qolish xavfi bor.   |

## 5. Abstraktsiya Tushunchasi: Murakkablikni Soddalashtirish San'ati

Abstraktsiya, SI sohasidagi eng muhim tushunchalardan biri bo'lib, murakkablikni kamaytirish uchun muhim xususiyatlarga e'tibor qaratib, keraksiz tafsilotlarni yashirish jarayonidir. Hisoblash nuqtai nazaridan, abstraktsiya hisoblash murakkabligini kamaytiruvchi, rasmiy tizimlar orasidagi "xaritalash" (mapping) sifatida belgilanadi. Bu texnika AI ga inson aqliga xos bo'lgan idrok, bilimni ifodalash va fikrlash kabi jarayonlarni takrorlashga yordam beradi.

Ushbu tushunchani tushunish uchun kundalik hayotdan bir qancha misollar keltirish mumkin. Masalan, kofe qaynatish mashinasidan foydalanishda siz uning ichki mexanizmlarini, masalan, suv nasosi yoki isitish elementining ishlashini tushunishingiz shart emas. Siz faqat yuqori darajadagi harakatlarni (suv va kofe solish, tugmani bosish) bilishingiz kifoya. Bu abstraktsiya tufayli murakkablik yashiriladi va vazifa soddalashtiriladi. Xuddi shunday, avtomobil boshqarayotganda siz dvigatelning har bir komponenti qanday ishlashini bilishingiz shart emas, balki faqat pedallar va rul yordamida harakatni boshqarasiz.

AI muammolarini hal qilishda abstraksiyaning amaliy roli bir necha muhim jihatlarida namoyon bo'ladi:

- **Hisoblash murakkabligini kamaytirish:** Abstraksiya katta holat maydoniga ega bo'lgan muammolarni samarali hal qilishda muhim rol o'ynaydi. Tizim keraksiz tafsilotlarni e'tibordan chetda qoldirib, faqat maqsadga ta'sir qiluvchi asosiy elementlarga e'tibor qaratadi, bu esa optimal yechimni topish uchun zarur bo'lgan hisob-kitoblar sonini keskin kamaytiradi.
- **Umumiy algoritmlarni yaratish:** Muammoga xos tafsilotlarni yashirish orqali, abstraksiya umumiy maqsadli AI doiralarini yaratishga imkon beradi. Bu esa turli muammolar uchun bir xil algoritmlarni (masalan, yo'l qidirish algoritmlarini) qo'llash imkonini beradi.

Shu bilan birga, zamonaviy AI tizimlarida abstraksiya inson-xos umumiy intellektga erishish yo'lidagi asosiy to'siqlardan birini ham ko'rsatadi. Hozirgi AI modellari, masalan, yirik til modellari (LLM), ko'pincha `pattern-matching` (naqshlarni aniqlash) ga asoslanadi. Masalan, "1 kg po'lat og'irimi yoki 1 kg patmi?" savoliga model to'g'ri javob berishi mumkin, chunki u bu savolni o'rgatilgan ma'lumotlarda minglab marta ko'rgan. Ammo agar savol biroz o'zgartirilsa ("10 kg po'lat og'irimi yoki 1 kg patmi?"), model xato qilish ehtimoli yuqori, chunki u naqshni yodlagan, ammo uning ortidagi asosiy fizik tamoyilni tushunmagan.

Boshqa tomondan, inson yangi, avval ko'rilmagan vaziyatlarda ham asosiy prinsiplarni (masalan, "og'irlik" yoki "ko'paytirish" tushunchalarini) qo'llay oladi. Bu, haqiqiy umumlashtirish (generalization) qobiliyatidir. Abstraksiya faqat hisoblash muammosini hal qilish usuli emas, balki bu Umumiy Sun'iy Intellekt (AGI) ga erishish yo'lidagi eng muhim muammolardan biridir. AI ga inson kabi umumlashtirish qobiliyatini o'rgatish, bugungi kunda sohaning eng muhim tadqiqot yo'nalishlaridan biri bo'lib qolmoqda.

## 6. Xulosa: Nazariyadan Amaliyotga

Ushbu hisobotda yoritilganidek, mantiqiy tizimlarda muammo va maqsadni shakllantirish, oddiy nazariy kontseptsiyadan ko'ra ko'proq narsa - bu AI ning muvaffaqiyatini belgilaydigan amaliy fandir. AI tizimining samaradorligi va kutilgan natijalarga erishishi, uning muammolarni qanchalik aniq va rasmiy shakllantira olishiga bevosita bog'liq. To'g'ri belgilangan maqsadlar, aniq rasmiy komponentlar (boshlang'ich holat, harakatlar, maqsadli sinov, yo'l narxi), optimal qidiruv algoritmlari va abstraksiya tamoyillari birgalikda agentning murakkab vaziyatlarda samarali harakat qilishini ta'minlaydi.

Bu elementlar o'zaro chambarchas bog'liq. Muammoni rasmiy shakllantirish bosqichi qidiruv daraxtining tuzilishini belgilaydi. Bu tuzilish esa, o'z navbatida, qaysi qidiruv algoritmi (masalan, BFS yoki DFS) eng samarali ekanligini aniqlaydi. Algoritmlarning o'zi ham o'ziga xos ma'lumot tuzilmalariga (navbat yoki stek) asoslanadi, bu esa ularning qidiruv strategiyasini (kenglik yoki chuqurlik bo'ylab) belgilaydi. Nihoyat, abstraksiya ushbu butun jarayonni soddalashtirib, keraksiz murakkabliklarni yashiradi, hisoblash yukini kamaytiradi va agentning umuman olganda ishlashini ta'minlaydi.

Xulosa qilib aytganda, mantiqiy tizimlarda muammoni shakllantirish faqatgina birinchi bosqich emas, balki bu butun yechim topish jarayonining asosi hisoblanadi. Bu asosiy tamoyillarni chuqur tushunish, AI ni hozirgi cheklangan qobiliyatlaridan inson kabi umumlashtira oladigan Umumiy Sun'iy Intellekt sari olib boruvchi ko'prikdir.

Sun'iy intellektda (AI) muammolarni shakllantirish - bu AI tizimi hal qila oladigan haqiqiy muammoni aniq belgilangan hisoblash muammosiga aylantirish jarayoni. Bu sun'iy intellektni rivojlantirishda muhim qadamdir, chunki u AI agenti optimal yechimga erishish uchun amalga oshirishi mumkin bo'lgan maqsadlar, cheklovlar va mumkin bo'lgan harakatlarni belgilaydi.

Muammoni to'g'ri shakllantirish orqali AI tizimlari o'z maqsadlariga erishish uchun qidiruv algoritmlarini, optimallashtirish usullarini va qaror qabul qilish modellarini samarali qo'llashi mumkin. AI yechimining samaradorligi ko'p jihatdan muammoning qanchalik to'g'ri aniqlanganiga bog'liq, chunki muammoni noto'g'ri shakllantirish samarasizlikka, noto'g'ri natijalarga yoki ortiqcha hisoblash murakkabligiga olib kelishi mumkin.

Masalan, sun'iy intellektga asoslangan marshrutni rejalashtirishda aqlli tizim o'zining dastlabki holatini (boshlang'ich joylashuvi), maqsad holatini (maqsadini), mavjud harakatlarni (bo'lishi kerak bo'lgan marshrutlarni) va optimallashtirish mezonlarini (eng qisqa masofa, eng kam tirbandlik va boshqalar) aniqlashi kerak. Muammoni samarali tuzib, sun'iy intellekt transport sharoitlari kabi real vaqt cheklovlariga moslashgan holda eng samarali marshrutni aniqlay oladi.

**Muammoni shakllantirish robototexnika, o'yin AI, avtonom tizimlar va aqlli qidiruv algoritmlarida** keng qo'llaniladi, bu uni AI tomonidan boshqariladigan qarorlarni qabul qilishning muhim jihatiga aylantiradi.

# Muammoni shakllantirishning asosiy komponentlari

AIDA muammolarni shakllantirish muammoning tuzilishini aniqlaydigan bir nechta asosiy komponentlardan iborat bo'lib, AI tizimlariga optimal echimlarni samarali aniqlash imkonini beradi. Ushbu komponentlar AI agentiga atrof-muhitni tushunishga, mumkin bo'lgan harakatlarni baholashga va oldindan belgilangan maqsadga erishishga yordam beradi.

## 1. Dastlabki holat

Dastlabki holat AI tizimining boshlang'ich nuqtasini ifodalaydi, u erdan qaror qabul qilish jarayoni boshlanadi. U muammoning kontekstini belgilaydigan asosiy ma'lumotlar yoki shartlarni taqdim etadi.

**Misol:** Shaxmat o'yinida boshlang'ich holat boshlang'ich taxta konfiguratsiyasi bo'lib, o'yin boshlanishidan oldin barcha donalar o'zlarining standart joylariga joylashtiriladi. AI mumkin bo'lgan harakatlar va strategiyalarni aniqlash uchun ushbu holatni tahlil qiladi.

## 2. Harakatlar to'plami (voris funksiyasi)

Voris funksiyasi sifatida ham tanilgan harakatlar to'plami AI ma'lum bir holatdan bajarishi mumkin bo'lgan barcha mumkin bo'lgan harakatlarni belgilaydi. Harakatlarning mavjudligi atrof-muhit sharoitlari va tizim cheklovlariga qarab o'zgaradi.

**Misol:** O'z-o'zidan boshqariladigan mashinada AI bir qator mumkin bo'lgan harakatlarga ega, masalan:

- Tezlikni oshirish uchun **tezlashtiring** .
- Sekinlashtirish yoki to'xtatish uchun **tormoz** .
- Chorrahalarda harakatlanish uchun **chapga yoki o'ngga buriling** .
- Yo'nalishni saqlab qolish uchun **chiziqda qoling** .

Har bir harakat AIning yo'li va qaror qabul qilishiga ta'sir qiladi va maqsadga erishishga hissa qo'shadi.

### 3. O'tish modeli

O'tish modeli harakatni amalga oshirgandan so'ng, AI qanday qilib bir holatdan ikkinchi holatga o'tishini tasvirlaydi. Bu turli harakatlar natijalarini bashorat qilishda yordam beradi va AIga ongli qarorlar qabul qilish imkonini beradi.

**Misol:** GPS navigatsiya tizimida foydalanuvchi marshrutni tanlaganda, AI avtomobil bir shahardan ikkinchisiga o'tayotganda holat o'tishlarini aniqlaydi. O'tish modeli qaror qabul qilishni takomillashtirish uchun masofa, yo'l sharoitlari va transport yangilanishlarini hisobga oladi.

### 4. Maqsad holati

Maqsad holati AI erishmoqchi bo'lgan istalgan yakuniy holatni yoki yechimni belgilaydi. Aniq belgilangan maqsadsiz, AI tizimi yo'nalishga ega emas va uning muvaffaqiyatini samarali baholay olmaydi.

**Misol:** Labirintni hal qiluvchi AIda maqsad holati chiqishga yetib boradi. AI mavjud yo'llarni qayta ishlaydi, to'siqlarni baholaydi va maqsadga erishishning eng samarali usulini belgilaydi.

### 5. Yo'l xarajati funksiyasi

Yo'l xarajati funksiyasi maqsadga erishish bilan bog'liq xarajatlarni o'lchaydi va AI eng samarali echimni tanlashini ta'minlaydi. Xarajat dasturga qarab vaqt, masofa, energiya iste'moli yoki hisoblash murakkabligi bilan o'lchanishi mumkin.

**Misol:** Google Xaritalarda AI bir nechta marshrutlarni ko'rib chiqadi va **eng qisqa yo'lni** quyidagi omillarga asoslanib baholaydi:

- Boshlanish va boradigan joy orasidagi **masofa** .
- Tirbandlikni oldini olish uchun **transport sharoitlari** .
- Eng tez marshrutni topish uchun **taxminiy sayohat vaqti** .

Ushbu asosiy komponentlarni birlashtirgan holda, AI tizimlari avtonom navigatsiyadan tortib strategik o'yin o'ynashgacha bo'lgan turli sohalaridagi muammolarni samarali shakllantirishi, tahlil qilishi va hal qilishi mumkin.



# Muammoni shakllantirish bosqichlari

AI da muammolarni shakllantirish jarayoni haqiqiy muammolarni AI tizimi hal qila oladigan aniq belgilangan hisoblash muammosiga aylantirishni o'z ichiga oladi. Bu muammoning ko'lamini aniqlashni, mumkin bo'lgan harakatlarni belgilashni va AI ni optimal echimga yo'naltirish uchun cheklovlarni o'rnatishni talab qiladi. Quyida muammoni shakllantirishning asosiy bosqichlari keltirilgan.

## 1-qadam: Muammo bayonotini aniqlang

Birinchi qadam, AI hal qilishi kerak bo'lgan muammoni aniq belgilashdir. Bu haqiqiy dunyo muammosini, uning maqsadlari va cheklovlarini aniqlashni o'z ichiga oladi. Yaxshi aniqlangan muammo AI agentlariga samarali qaror qabul qilish uchun tegishli algoritmlarni qo'llashga yordam beradi.

**Misol:** AI ga asoslangan paketlarni yetkazib berish tizimida muammo bayoni quyidagicha bo'lishi mumkin:

- Paketni **A omboridan mijoz B ga** yetkazing .
- **Yetkazib berish muddati va yoqilg'i sarfini** minimallashtiring .
- **Yo'l harakati sharoitlari va ob-havo o'zgarishlariga** moslash .

Muammoni aniq belgilash AI ga yechimga ta'sir qiluvchi o'zgaruvchilar, bog'liqliklar va cheklovlarni aniqlash imkonini beradi.

## 2-qadam: Dastlabki holat va maqsad holatini o'rnatish

Muammo aniqlangandan so'ng, AI quyidagilarni aniqlashi kerak:

- **Dastlabki holat** : Tizimning ishga tushirish shartlari.
- **Maqsad holati** : AI tomonidan boshqariladigan harakatlar bajarilgandan so'ng kerakli natija.

Paketni yetkazib berish AI uchun:

- **Dastlabki holat - A omboridagi paket** .
- **Maqsad holati B mijoziga** eng qisqa vaqt ichida muvaffaqiyatli yetkazib berishdir.

Ushbu holatlarni aniqlash sun'iy intellektga boshlang'ich sharoitlarni tushunishga va maqsadga erishilganda baholashga yordam beradi.

## 3-qadam: Mavjud harakatlar va holatga o'tish modelini aniqlang

Keyinchalik, AI qanday harakatlar qilishi mumkinligini va bu harakatlar muammo holatiga qanday ta'sir qilishini aniqlashi kerak.

- **Mavjud harakatlar** : AIning real sharoitlarga javoban mumkin bo'lgan harakatlari.
- **Holatga o'tish modeli** : Har bir harakat AI holatini qanday o'zgartiradi.

Paket yetkazib berish AI uchun harakatlar quyidagilarni o'z ichiga oladi:

- **Oldinga harakatlaning** (keyingi yo'l segmentiga o'tish).
- **Chapga yoki o'ngga buriling** (chorrahalarida yo'nalishni o'zgartiring).
- **To'xtating** (svetofor yoki mijozning tasdiqlanishini kuting).

O'tish modeli AIga tanlangan harakatlar asosida keyingi holatni bashorat qilishda yordam beradi, muammoni mantiqiy va samarali hal qilishni ta'minlaydi.

## 4-qadam: Cheklovlar va yo'l narxini aniqlang

Yakuniy bosqich - bu AI qarorlarini qabul qilishga yordam beradigan cheklovlar va optimallashtirish mezonlarini aniqlash.

- **Cheklovlar** : Mavjud echimlarni cheklovchi shartlar (masalan, yo'l harakati qoidolari, paketning og'irligi chegaralari yoki etkazib berish muddatlari).
- **Yo'l xarajati funktsiyasi** : Eng kam xarajat (masalan, eng qisqa yo'l, eng kam yoqilg'i sarfi) asosida eng maqbul echimni aniqlash uchun foydalaniladigan ko'rsatkich.

Paket yetkazib berish AI uchun optimallashtirish omillari quyidagilarni o'z ichiga olishi mumkin:

- **Belgilangan joyga eng qisqa masofa** .
- **Minimal yoqilg'i sarfi** .
- **Tezroq yetkazib berish uchun tirband yo'llardan qoching** .

Ushbu bosqichlarni bajarish orqali AI tizimlari muammolarni hal qilish yondashuvlarini tizimli ravishda tahlil qilishi, tuzilishi va optimallashtirishi mumkin, bu esa samaradorlik, aniqlik va avtomatlashtirishni yaxshilashga olib keladi.

## Alda muammolarni shakllantirish misoli: Avtonom paketlarni yetkazib berish

Avtonom paketlarni yetkazib berish - bu haqiqiy sun'iy intellekt ilovasi bo'lib, unda AI tizimi yoqilg'i samaradorligi, yetkazib berish vaqti va yo'l sharoitlari kabi omillarni hisobga olgan holda optimal yetkazib berish yo'nalishini rejalashtiradi va amalga oshiradi. Quyida biz ushbu muammoni har bir asosiy komponentni ko'rsatadigan kod parchalari, so'ngra yakuniy amalga oshirish bilan shakllantiramiz.

### Dastlabki holat

Dastlabki holat AI tizimining har qanday harakat qilishdan oldin boshlang'ich shartlarini ifodalaydi.

- Paket **A omborida** , jo'natishga tayyor.

- Avtonom **etkazib berish vositasi omborda joylashgan bo'lib** , marshrut ko'rsatmalarini kutmoqda.
- ```
class DeliveryAgent:
```

```
    def __init__(self):
```

```
        self.location = "Warehouse A" # Initial state
```

```
        self.destination = "Destination B"
```

```
        self.path = [] # Stores the path taken by the agent
```

```
    def get_current_state(self):
```

```
        return f"Package is currently at {self.location}"
```

## Harakatlar to'plami

Harakatlar to'plami o'z maqsadiga erishish uchun AIning mumkin bo'lgan harakatlarini belgilaydi.

### Mavjud harakatlar:

- **Oldinga siljiting** - joriy yo'nalishda davom eting.
  - **Chapga buriling** - chorrahada yo'nalishni o'zgartiring.
  - **O'ngga buriling** - muqobil marshrutga o'ting.
  - **To'xtatish** - To'siq yoki etkazib berishni tasdiqlash tufayli harakatni to'xtatish.
- ```
class Actions:
```

```
    @staticmethod
```

```
def move_forward(agent):
```

```
    agent.path.append("Move Forward")
```

```
    return "Moving Forward"
```

```
@staticmethod
```

```
def turn_left(agent):
```

```
    agent.path.append("Turn Left")
```

```
    return "Turning Left"
```

```
@staticmethod
```

```
def turn_right(agent):
```

```
    agent.path.append("Turn Right")
```

```
    return "Turning Right"
```

```
@staticmethod
```

```
def stop(agent):
```

```
    agent.path.append("Stop")
```

```
    return "Stopping"
```

## O'tish modeli

O'tish modeli bajarilgan harakatlar asosida tizimning bir holatdan ikkinchi holatga o'tishini tavsiflaydi.

- Agar AI **oldinga siljitishni** tanlasa , tizim **joriy GPS manzilini** yangilaydi .
- Agar sun'iy intellekt **chapga yoki o'ngga burilishni** tanlasa, u o'z sarlavhasi va **yo'nalishini** mos ravishda yangilaydi .

```
class TransitionModel:
```

```
    @staticmethod
```

```
    def update_state(agent, action):
```

```
        if action == "Move Forward":
```

```
            agent.location = "Next Location"
```

```
        elif action == "Turn Left":
```

```
            agent.location = "Left Turn Location"
```

```
elif action == "Turn Right":
```

```
    agent.location = "Right Turn Location"
```

```
    return f"New location: {agent.location}"
```

## Maqsad holati

Maqsad holati AI vazifasining muvaffaqiyatli bajarilishini belgilaydi.

- **Paket B manziliga yetkaziladi.**
  - **Vaqt va yoqilg'i samaradorligi optimallashtirilgan.**
- ```
def check_goal_state(agent):
```

```
    return agent.location == agent.destination
```

## Yo'l narxi funktsiyasi

Yo'l xarajati funktsiyasi turli yo'nalishlarning samaradorligini baholaydi va optimal qaror qabul qilishni ta'minlaydi.

- **Yoqilg'i tejamkorligi** - energiya sarfi past bo'lgan marshrutlarni tanlash.
  - **Etkazib berish muddati** - keraksiz aylanmalardan qochib, eng tez yo'lga ustunlik berish.
- ```
def path_cost_function(distance, fuel_usage, traffic_delay):
```

```
    return distance * fuel_usage + traffic_delay # Example cost function
```

## Avtonom paketlarni yetkazib berish Alning yakuniy amalga oshirilishi

Endi biz barcha komponentlarni yakuniy AIga asoslangan muammoni shakllantirish va simulyatsiya qilishga birlashtiramiz.

```
import random
```

```
class DeliveryAgent:
```

```
    def __init__(self):
```

```
        self.location = "Warehouse A" # Initial state
```

```
        self.destination = "Destination B"
```

```
        self.path = [] # Stores the path taken
```

```
        self.fuel_usage = 0.5 # Fuel per distance unit
```

```
        self.total_cost = 0
```

```
    def get_current_state(self):
```

```
        return f"Package is currently at {self.location}"
```

```
class Actions:
```

```
    @staticmethod
```

```
def move_forward(agent):
```

```
    agent.path.append("Move Forward")
```

```
    agent.location = "Next Location"
```

```
    return "Moving Forward"
```

```
@staticmethod
```

```
def turn_left(agent):
```

```
    agent.path.append("Turn Left")
```

```
    agent.location = "Left Turn Location"
```

```
    return "Turning Left"
```

```
@staticmethod
```

```
def turn_right(agent):
```

```
    agent.path.append("Turn Right")
```



```
agent.location = "Right Turn Location"
```

```
return "Turning Right"
```

```
@staticmethod
```

```
def stop(agent):
```

```
agent.path.append("Stop")
```

```
return "Stopping"
```

```
class TransitionModel:
```

```
@staticmethod
```

```
def update_state(agent, action):
```

```
if action == "Move Forward":
```

```
agent.location = "Next Location"
```

```
elif action == "Turn Left":
```

```
    agent.location = "Left Turn Location"
```

```
elif action == "Turn Right":
```

```
    agent.location = "Right Turn Location"
```

```
return f"New location: {agent.location}"
```

```
def check_goal_state(agent):
```

```
    return agent.location == agent.destination
```

```
def path_cost_function(distance, fuel_usage, traffic_delay):
```

```
    return distance * fuel_usage + traffic_delay
```

```
def delivery_simulation():
```

```
    agent = DeliveryAgent()
```

```
    print("Starting Delivery Simulation...")
```

```
print(agent.get_current_state())
```

```
actions = [Actions.move_forward, Actions.turn_left, Actions.turn_right, Actions.stop]
```

```
for _ in range(5): # Simulating 5 steps
```

```
    action = random.choice(actions)
```

```
    print(action(agent))
```

```
    print(TransitionModel.update_state(agent, action.__name__))
```

```
    if check_goal_state(agent):
```

```
        print("Package Delivered Successfully!")
```

```
        break
```

```
total_cost = path_cost_function(10, agent.fuel_usage, random.randint(1, 5))
```

```
print(f"Total Delivery Cost: {total_cost}")
```

# Run the simulation

delivery\_simulation()

# Alda muammolarni shakllantirishning ahamiyati

Muammoni shakllantirish AIda juda muhim, chunki u murakkab muammolarni hisoblash uchun echiladigan modellarga tuzadi, samaradorlikni oshiradi va qaror qabul qiladi. Yaxshi ishlab chiqilgan muammo AI tizimlariga ma'lumotlarni mantiqiy qayta ishlash, qidiruv jarayonlarini optimallashtirish va hisoblash murakkabligini kamaytirish imkonini beradi.

Asosiy afzalliklarga quyidagilar kiradi:

- **Yaxshilangan sun'iy intellekt samaradorligi** - muammoni aniq tuzish xatolarni kamaytiradi va ish faoliyatini yaxshilaydi.
- **Kamaytirilgan hisoblash murakkabligi** - Optimallashtirilgan qaror qabul qilish keraksiz resurslarni sarflashning oldini oladi.
- **Yaxshiroq optimallashtirish** – sun'iy intellekt asosidagi yechimlar real hayotdagi ilovalarda tezroq va tejamkorroq natijalarga erishadi.

Maqsadlar, cheklovlar va holat o'tishlarini samarali belgilash orqali AI muammolarni aniq hal qilishi, o'zgaruvchan muhitga moslashishi va robototexnika, logistika va aqlli qidiruv kabi sohalarda avtomatlashtirishni yaxshilashi mumkin.

## Muammoni shakllantirishdagi qiyinchiliklar

AIda muammolarni shakllantirish aniqlik, samaradorlik va qaror qabul qilishga ta'sir qiluvchi turli qiyinchiliklar bilan birga keladi. Ba'zi asosiy qiyinchiliklarga quyidagilar kiradi:

1. **Haqiqiy dunyo muammolarining murakkabligi** - AI noaniqlik, dinamik muhit va to'liq bo'lmagan ma'lumotlarni boshqarishi kerak, bu esa muammolarni tuzishni qiyinlashtiradi.
2. **To'g'ri maqsadni aniqlash** - Ba'zi AI ilovalari qarama-qarshi maqsadlarni o'z ichiga oladi (masalan, avtonom transport vositalarida tezlik va yoqilg'i samaradorligini muvozanatlash).
3. **Hisoblash cheklovlari** - AI tomonidan boshqariladigan tizimlar ko'pincha katta hajmdagi ishlov berish quvvatini talab qiladi, bu esa optimallashtirishni juda muhim qiladi.
4. **Axloqiy mulohazalar** – sog'liqni saqlash, moliya va qonunchilikdagi sun'iy intellekt mas'uliyatli qarorlar qabul qilishni ta'minlash uchun adolat, maxfiylik va oshkoralik qoidalariga amal qilishi kerak.

## Xulosa

Muammoni shakllantirish AI qarorlarini qabul qilishning asosiy jihatini hisoblanadi, chunki u AI tomonidan boshqariladigan echimlarni boshqaradigan tuzilma, cheklovlar va maqsadlarni belgilaydi. Muammolarni aniq shakllantirish orqali AI tizimlari qidiruv jarayonlarini optimallashtirishi, samaradorlikni oshirishi va qaror qabul qilishning murakkab stsenariylarini boshqarishi mumkin.

Uning ilovalari robototexnika, avtonom tizimlar, aqlli qidiruv va mashinani o'rganish bo'yicha keng qamrovli bo'lib, sun'iy intellektga o'zi boshqariladigan avtomobillar, avtomatlashtirilgan logistika va strategik rejalashtirish kabi sohalarda real muammolarni hal qilish imkonini beradi.

Uning afzalliklariga qaramay, noaniqlik, hisoblash cheklovlari va axloqiy tashvishlar kabi muammolar keyingi tadqiqotlar zarurligini ta'kidlaydi. AI muammolarini shakllantirish usullarini optimallashtirish kelajakdagi ilovalar uchun yanada moslashuvchan, samarali va axloqiy AI yechimlariga olib keladi.